

Agentic Commerce: Technical Architecture

Part 2: Protocol Specifications for Implementation

A three-protocol architecture enabling trustworthy, decentralized commerce between AI agents

[!NOTE] **About This Document**

This document series is an independent architectural exploration and reference implementation built upon Google's UCP specification and complementary open protocols (MCP, A2A, W3C DIDs/VCs). This is not an official Google publication. Community feedback and collaboration is welcome

What this document is NOT:

- An official Google publication or endorsed implementation
- A production-ready specification
- A commercial product

Introduction

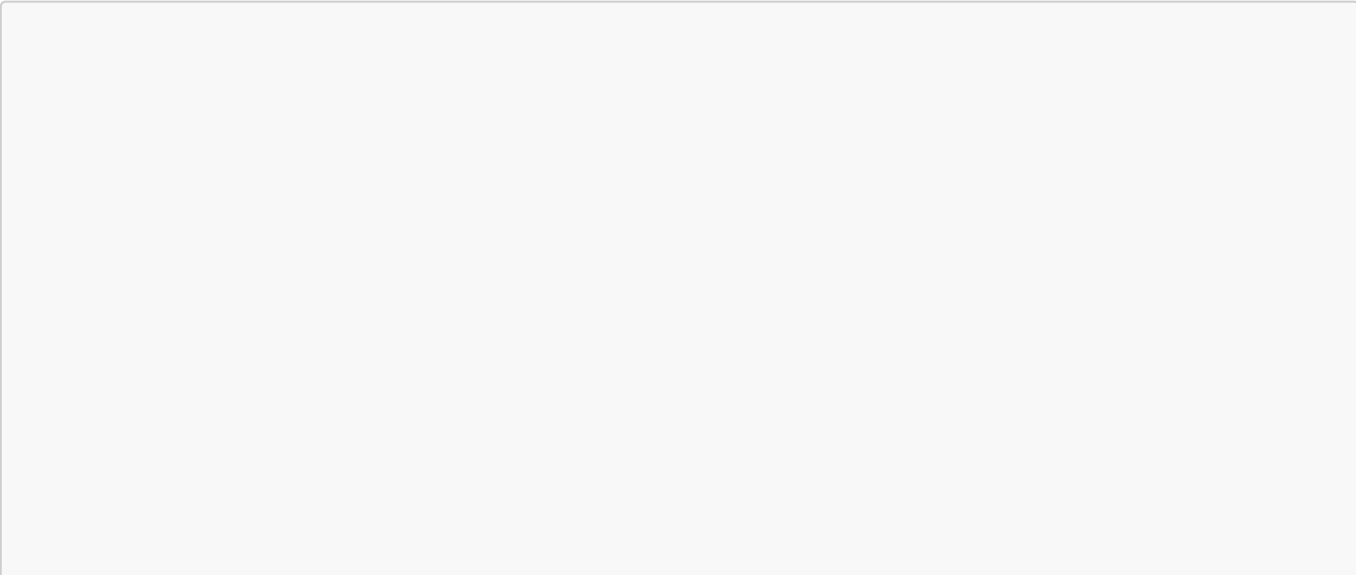
This document provides the technical architecture for Agentic Commerce—a framework where AI agents conduct commerce transactions on behalf of users through open, decentralized protocols.

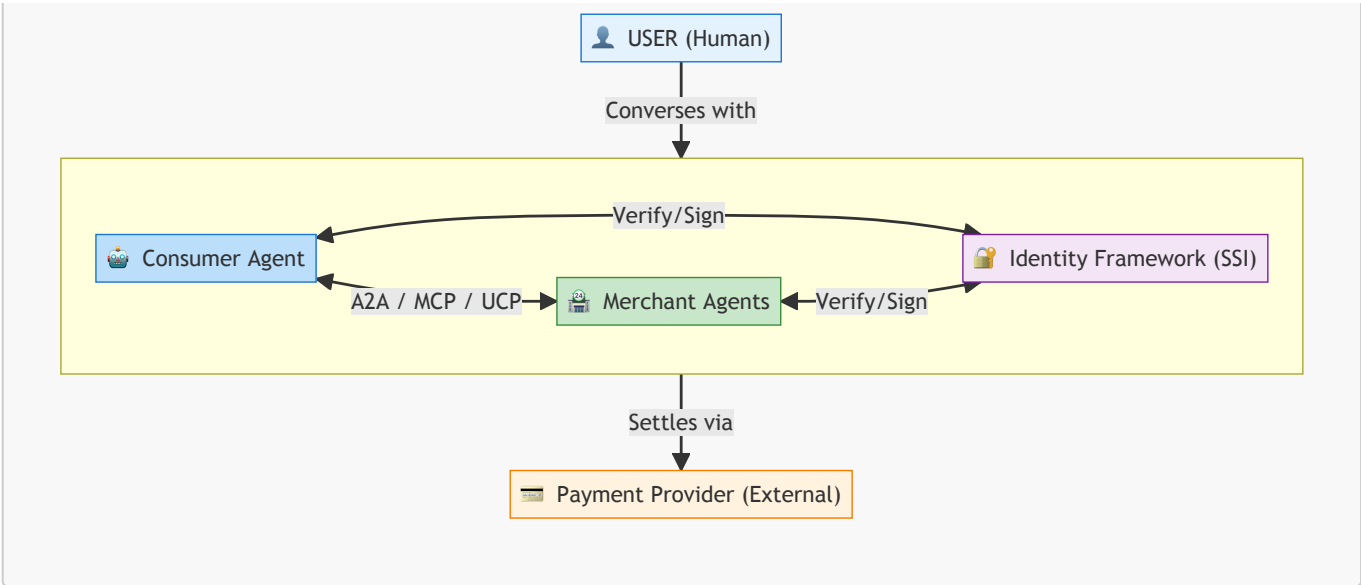
This is Part 2 of the Agentic Commerce whitepaper series:

- **Part 1:** Concept & Vision (AGENTIC_COMMERCE_PART1_CONCEPT.md) — The problem statement and high-level approach
- **Part 2:** Technical Architecture (this document) — Protocol specifications for implementation
- **Part 3:** (Coming soon) — Reference implementation walkthrough

System Context

Before diving into protocol details, here's the high-level system context:





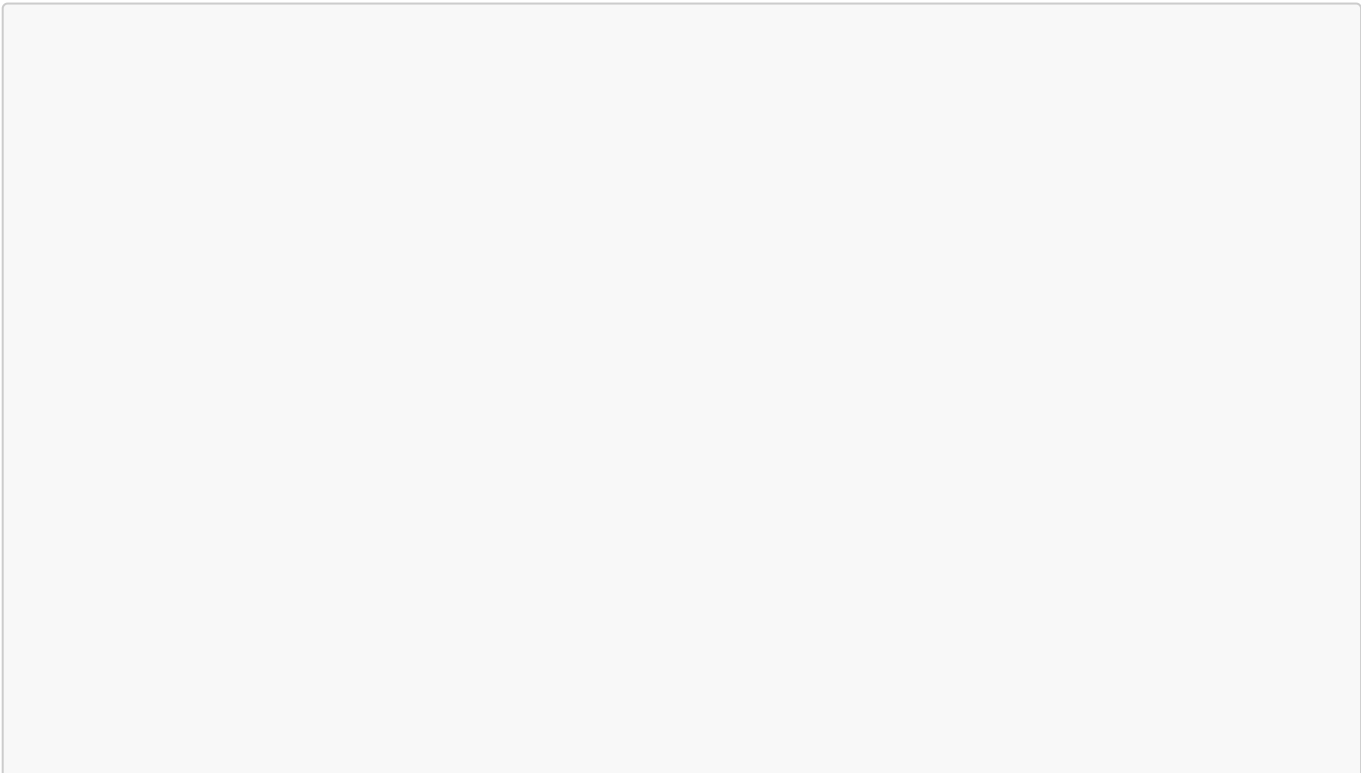
Agentic Commerce Ecosystem: Consumer and Merchant agents communicate via **A2A** (agent collaboration), **MCP** (tool discovery), and **UCP** (commerce workflow). Both verify identities through the **SSI** framework.

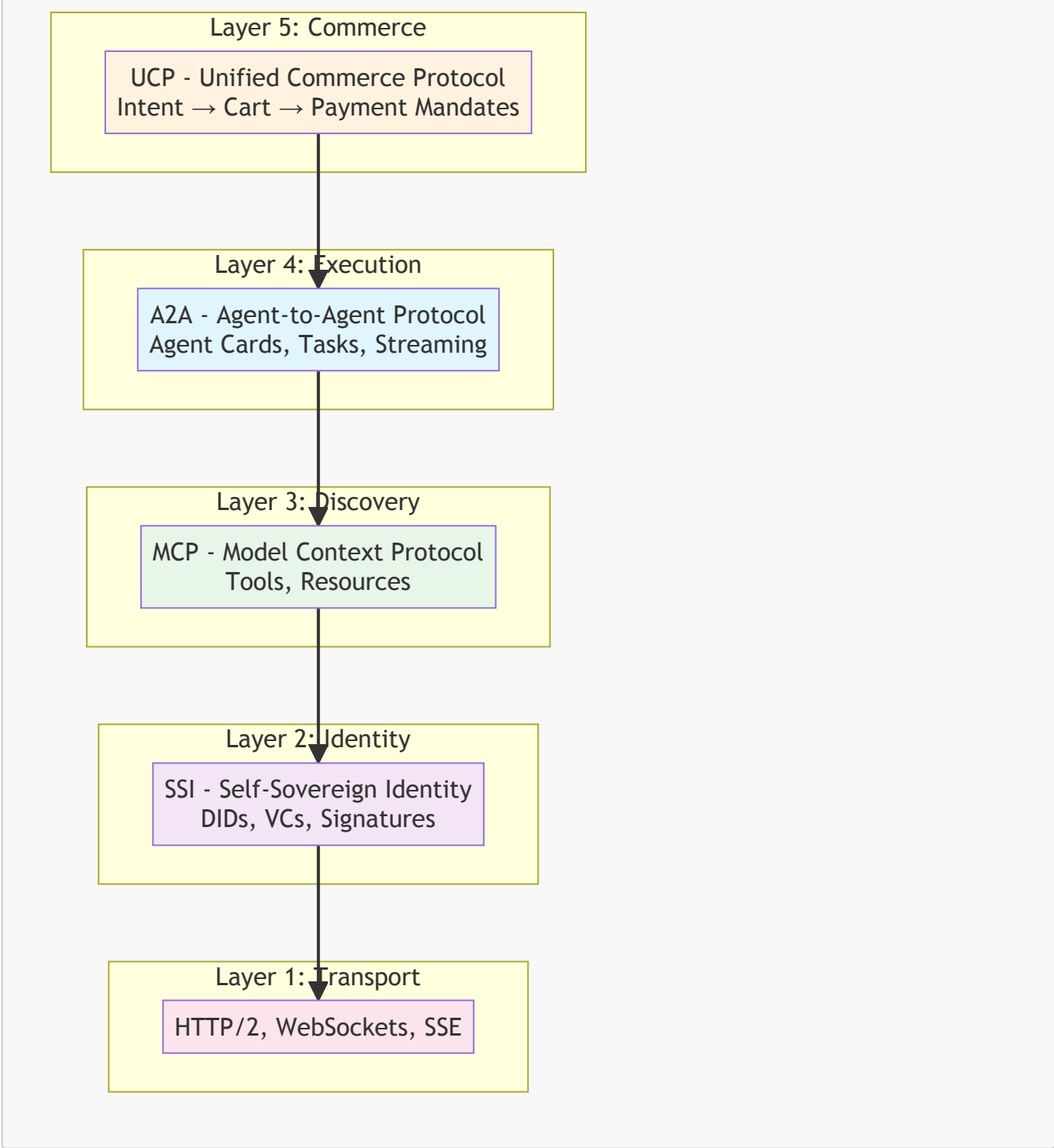
Participants:

- **User:** Human who delegates shopping tasks to their AI agent
- **Consumer Agent:** AI assistant that discovers products and conducts purchases
- **Merchant Agents:** AI agents representing sellers, handling orders and fulfillment
- **Identity Framework:** Shared SSI infrastructure for DIDs, VCs, and mandate signing
- **Payment Provider:** External payment rails (Stripe, crypto, etc.)

Protocol Stack Overview

The architecture uses a layered protocol stack where each protocol serves a specific purpose:

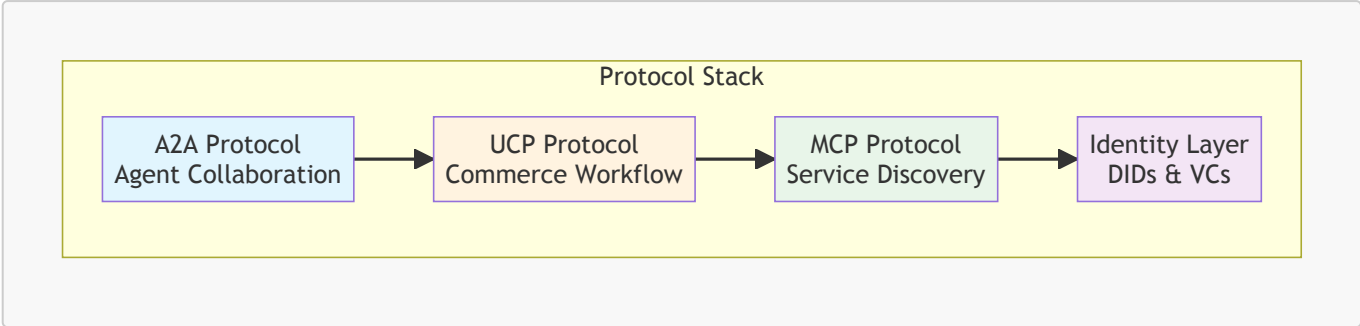




Protocol Purposes

Layer	Protocol	Purpose	Examples
5 - Commerce	UCP	Commerce-specific workflow	Intent → Cart → Payment mandates
4 - Execution	A2A	Agent-to-agent transactions	Order processing, negotiation
3 - Discovery	MCP	Read-only queries to services	product_search, get_pricing
2 - Identity	SSI	Trust & verification	verify_did, sign_mandate
1 - Transport	HTTP	Network communication	Request/Response, Streaming

Protocol Interaction Diagram



1. Identity Layer: DIDs and Verifiable Credentials

1.1 Decentralized Identifiers (DIDs)

Every participant in Agentic Commerce—users, merchants, agents—has a Decentralized Identifier. DIDs are:

- **Self-sovereign:** Controlled by the entity, not a platform
- **Globally unique:** No collisions across the ecosystem
- **Cryptographically verifiable:** Anyone can verify signatures without a central authority

1.2 DID Methods

The architecture is DID method-agnostic. Different use cases call for different approaches:

Method	Anchor	Best For	Resolution
did:web	Domain DNS	Enterprises, established merchants	HTTP lookup
did:key	Self-contained	Ephemeral agents, quick setup	Decode from DID itself
did:ethr	Ethereum blockchain	Maximum decentralization	On-chain lookup

did:web Example (for a merchant):

```
did:web:electronics-store.com
```

Resolves via: <https://electronics-store.com/.well-known/did.json>

did:key Example (for a consumer agent):

```
did:key:z6MkhaXgBZDvotDkL5257faiztiGiC2QtKLGpbnnEGta2doK
```

Public key embedded in the identifier itself—no resolution needed.

did:ethr Example (for blockchain-anchored identity):

```
did:ethr:sepolia:0x1a2b3c4d5e6f7890abcdef1234567890abcdef12
```

Resolves via Ethereum Sepolia network.

1.3 DID Document Structure

```
{
  "@context": ["https://www.w3.org/ns/did/v1"],
  "id": "did:web:merchant.example.com",
  "verificationMethod": [{
    "id": "did:web:merchant.example.com#key-1",
    "type": "EcdsaSecp256k1VerificationKey2019",
    "controller": "did:web:merchant.example.com",
    "publicKeyHex": "04a1b2c3d4e5f6..."
  }],
  "authentication": ["did:web:merchant.example.com#key-1"],
  "assertionMethod": ["did:web:merchant.example.com#key-1"],
  "service": [{
    "id": "did:web:merchant.example.com#mcp",
    "type": "MCPServer",
    "serviceEndpoint": "https://api.merchant.example.com/mcp"
  }]
}
```

1.4 Verifiable Credentials (VCs)

All UCP mandates are packaged as W3C Verifiable Credentials:

```
{
  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    "https://ucp.protocol/v1"
  ],
  "id": "urn:uuid:3f5c8a2e-1234-5678-9abc-def012345678",
  "type": ["VerifiableCredential", "CartMandate"],
  "issuer": "did:web:merchant.example.com",
  "issuanceDate": "2026-02-08T12:00:00Z",
  "credentialSubject": {
    // Mandate-specific claims
  },
  "proof": {
    "type": "EcdsaSecp256k1Signature2019",
    "created": "2026-02-08T12:00:00Z",
    "verificationMethod": "did:web:merchant.example.com#key-1",
    "proofPurpose": "assertionMethod",
    "jws": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0..."
  }
}
```

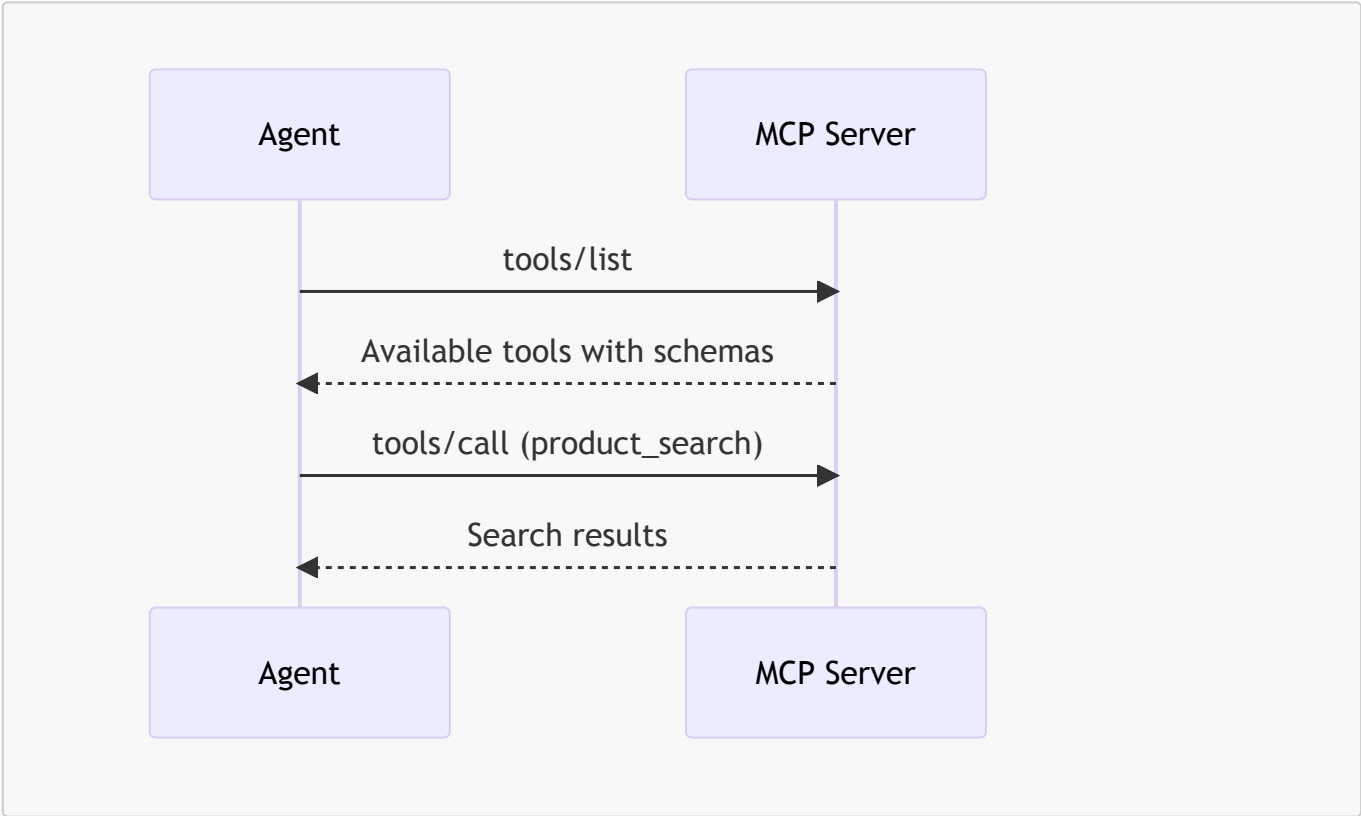
2. MCP: Model Context Protocol

MCP is the transport layer enabling AI agents to discover and interact with services.

2.1 Core Concepts

Concept	Description
Server	Service exposing capabilities via MCP
Client	Agent that discovers and calls MCP servers
Tools	Functions the agent can invoke
Resources	Data endpoints the agent can query

2.2 Tool Discovery



Request: List Tools

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}
```

Response: Tool Definitions

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "product_search",
        "description": "Search products in the catalog",
        "inputSchema": {
          "type": "object",
          "properties": {
            "query": {"type": "string"},
            "max_price": {"type": "number"},
            "category": {"type": "string"}
          },
          "required": ["query"]
        }
      },
      {
        "name": "ucp_intent",
        "description": "Submit purchase intent (UCP Stage 1)",
        "inputSchema": {
          "type": "object",
          "properties": {
            "intent_vc": {"type": "object", "description": "Signed Intent
Mandate VC"}
          },
          "required": ["intent_vc"]
        }
      }
    ]
  }
}
```

2.3 UCP Tools via MCP

Merchants expose UCP capabilities as MCP tools:

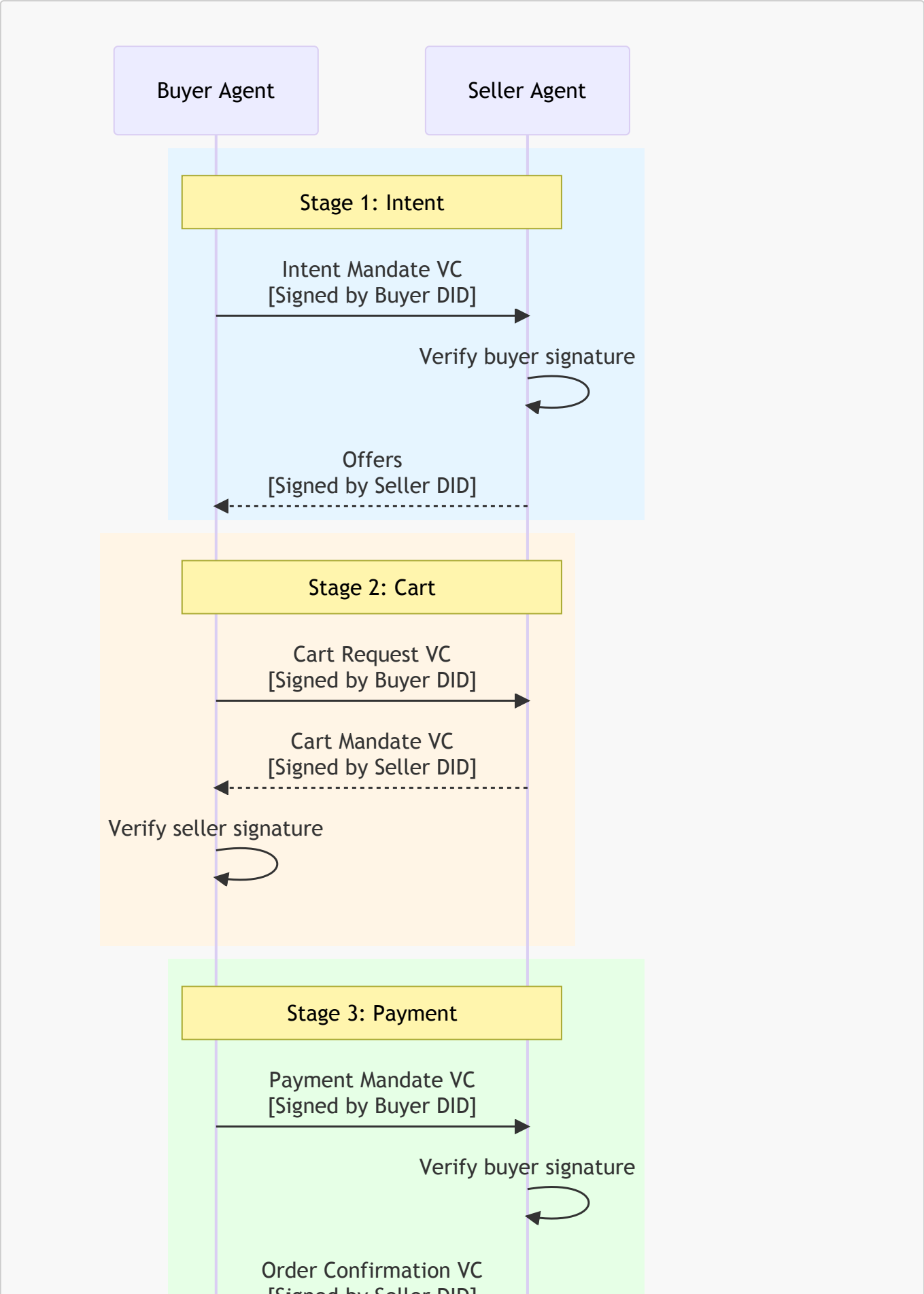
Tool Name	UCP Stage	Purpose
ucp_intent	Stage 1	Submit purchase intent
create_cart_mandate	Stage 2	Request locked pricing
submit_payment_mandate	Stage 3	Authorize payment

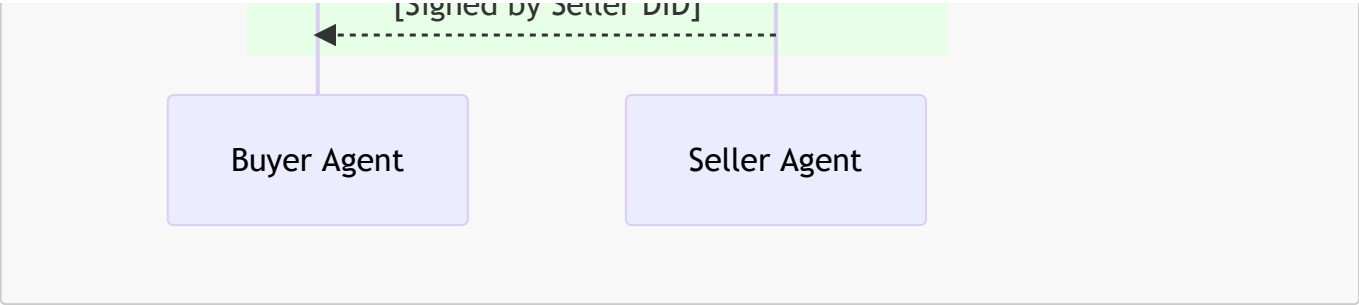
This design lets any MCP-speaking agent participate in UCP commerce without UCP-specific client code.

3. UCP: Unified Commerce Protocol

UCP defines the commerce workflow through three cryptographically signed mandates.

3.1 Protocol Flow





3.2 Mandate Schemas

Intent Mandate (Stage 1)

Buyer declares intent to purchase.

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1",
    "https://ucp.protocol/v1"],
  "id": "urn:uuid:intent-abc123",
  "type": ["VerifiableCredential", "IntentMandate"],
  "issuer": "did:key:z6MkConsumer...",
  "issuanceDate": "2026-02-08T12:00:00Z",
  "credentialSubject": {
    "action": "purchase_intent",
    "query": "laptop for development",
    "items": [
      {"product_id": "LAPTOP-001", "quantity": 1}
    ],
    "constraints": {
      "max_price": 1500,
      "currency": "USD"
    },
    "timestamp": 1707400800
  },
  "proof": { /* ... */ }
}
```

Field	Type	Required	Description
action	string	Yes	Always "purchase_intent"
query	string	No	Natural language description
items	array	No	Specific products if known
constraints.max_price	number	No	Budget limit
constraints.currency	string	No	ISO 4217 currency code

Cart Mandate (Stage 2)

Merchant provides binding offer with locked pricing.

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1",
    "https://ucp.protocol/v1"],
  "id": "urn:uuid:cart-mandate-xyz789",
  "type": ["VerifiableCredential", "CartMandate"],
  "issuer": "did:web:merchant.example.com",
  "issuanceDate": "2026-02-08T12:05:00Z",
  "credentialSubject": {
    "cart_id": "CART-ABC12345",
    "intent_ref": "urn:uuid:intent-abc123",
    "consumer_did": "did:key:z6MkConsumer...",
    "items": [
      {
        "product_id": "LAPTOP-PRO-16",
        "name": "ProBook 16 Developer Edition",
        "quantity": 1,
        "unit_price": 1299.00,
        "line_total": 1299.00
      }
    ],
    "subtotal": 1299.00,
    "tax": 103.92,
    "total": 1402.92,
    "currency": "USD",
    "valid_until": "2026-02-08T12:35:00Z",
    "terms": {
      "return_policy": "30-day returns",
      "warranty": "1 year manufacturer warranty"
    }
  },
  "proof": { /* ... */ }
}
```

Field	Type	Required	Description
cart_id	string	Yes	Unique cart identifier
intent_ref	string	Yes	Reference to original Intent
consumer_did	string	Yes	Buyer's DID
items	array	Yes	Line items with locked prices
total	number	Yes	Total amount due
valid_until	datetime	Yes	Expiration timestamp
terms	object	No	Additional terms

Payment Mandate (Stage 3)

Buyer authorizes payment.

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1",
    "https://ucp.protocol/v1"],
  "id": "urn:uuid:payment-mandate-def456",
  "type": ["VerifiableCredential", "PaymentMandate"],
  "issuer": "did:key:z6MkConsumer...",
  "issuanceDate": "2026-02-08T12:10:00Z",
  "credentialSubject": {
    "cart_mandate_ref": "urn:uuid:cart-mandate-xyz789",
    "cart_id": "CART-ABC12345",
    "authorized_amount": 1402.92,
    "currency": "USD",
    "payment_method": "card",
    "shipping_address": {
      "name": "Jane Developer",
      "line1": "456 Code Street",
      "city": "San Francisco",
      "state": "CA",
      "postal_code": "94102",
      "country": "US"
    }
  },
  "proof": { /* ... */ }
}
```

Order Confirmation

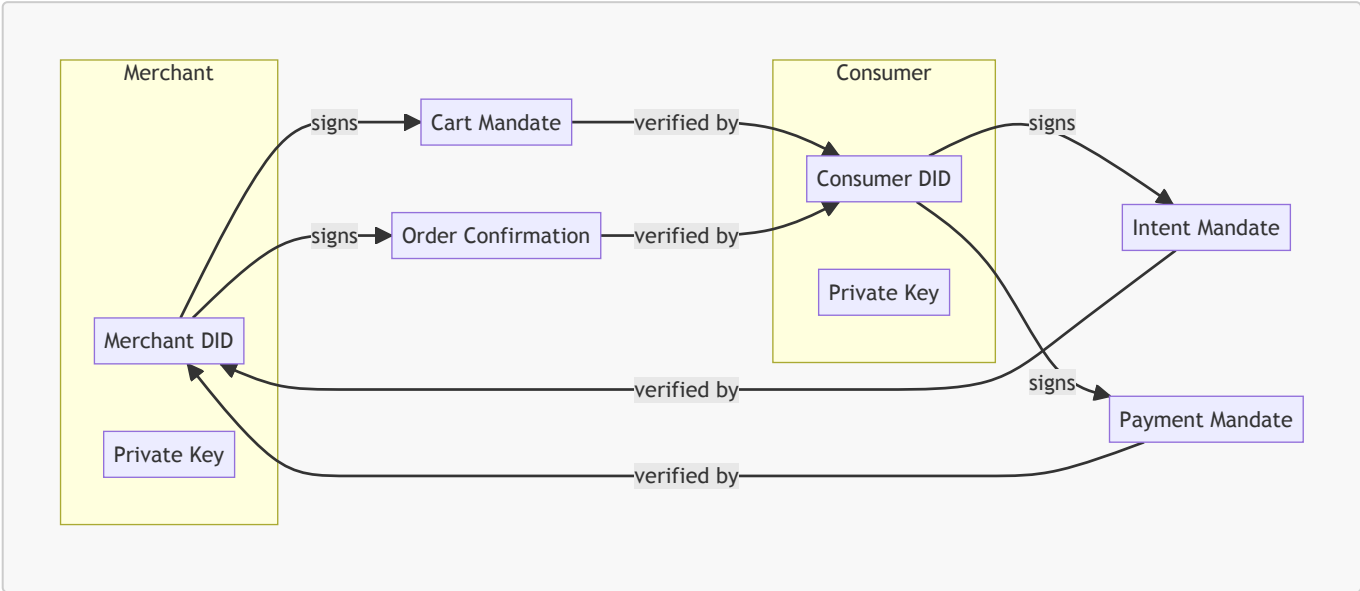
Merchant confirms order placement.

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1",
    "https://ucp.protocol/v1"],
  "id": "urn:uuid:order-conf-ghi789",
  "type": ["VerifiableCredential", "OrderConfirmation"],
  "issuer": "did:web:merchant.example.com",
  "issuanceDate": "2026-02-08T12:15:00Z",
  "credentialSubject": {
    "order_id": "ORD-2026020812345",
    "cart_mandate_ref": "urn:uuid:cart-mandate-xyz789",
    "payment_mandate_ref": "urn:uuid:payment-mandate-def456",
    "status": "confirmed",
    "amount_charged": 1402.92,
    "currency": "USD",
    "estimated_delivery": "2026-02-12",
    "tracking": {
      "carrier": "FedEx",
      "tracking_number": "1234567890"
    }
  },
}
```

```
"proof": { /* ... */ }
}
```

3.3 Signature Verification

Every mandate exchange includes bidirectional verification:



Trust Properties:

- **Non-repudiation:** Signatures prove who issued each mandate
- **Integrity:** Tampering invalidates signatures
- **Binding:** Cart prices are locked by merchant signature
- **Authorization:** Payment is explicitly authorized by buyer signature

4. A2A: Agent-to-Agent Protocol

A2A enables direct communication between agents for complex, multi-party workflows.

4.1 Agent Cards

Each agent publishes a discoverable capability description:

```
{
  "name": "Electronics Merchant Agent",
  "description": "AI agent for electronics commerce",
  "url": "https://agent.merchant.example.com",
  "version": "1.0.0",
  "provider": {
    "organization": "Merchant Inc.",
    "url": "https://merchant.example.com"
  },
  "capabilities": {
    "streaming": false,
    "pushNotifications": true,
  }
}
```

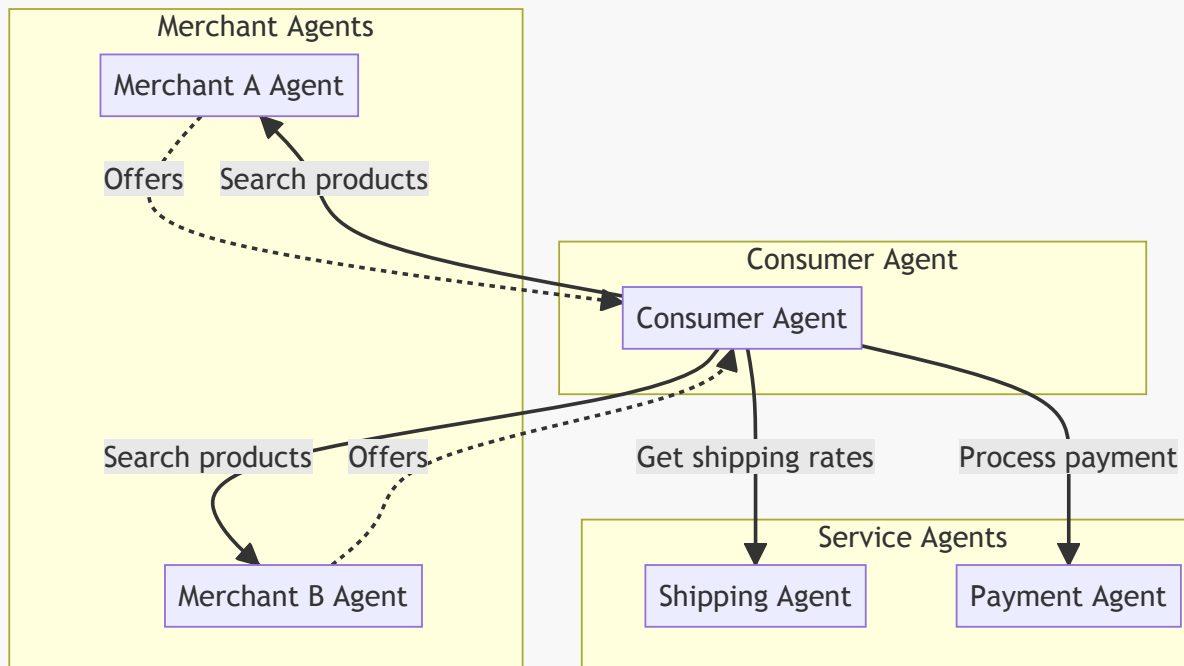
```

    "stateTransitionHistory": true
  },
  "authentication": {
    "schemes": ["bearer", "did-auth"]
  },
  "defaultInputModes": ["text"],
  "defaultOutputModes": ["text"],
  "skills": [
    {
      "id": "product_search",
      "name": "Product Search",
      "description": "Search electronics products",
      "tags": ["commerce", "search", "electronics"]
    },
    {
      "id": "ucp_commerce",
      "name": "UCP Commerce",
      "description": "Full UCP purchase flow support",
      "tags": ["commerce", "ucp", "purchase"]
    }
  ]
}

```

Discovery: Agent cards are published at [/.well-known/agent.json](#)

4.2 Task Delegation



Task Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tasks/send",
  "params": {
    "id": "task-12345",
    "message": {
      "role": "user",
      "parts": [{
        "type": "text",
        "text": "Find laptops under $1500 with 32GB RAM"
      }]
    },
    "sessionId": "session-abc123"
  }
}
```

5. Security Model

5.1 Trust Properties

Property	Mechanism
Authentication	DID-based identity for all parties
Integrity	Cryptographic signatures on all mandates
Non-repudiation	Signed VCs create unforgeable audit trail
Authorization	Each mandate explicitly authorizes next step
Privacy	Peer-to-peer, no central observer

5.2 Attack Mitigations

Attack	Mitigation
Impersonation	Signature verification against DID
Replay attacks	Unique mandate IDs, timestamps, expiry
Price manipulation	Locked prices in signed Cart Mandate
Unauthorized payment	Payment references specific Cart ID
Man-in-the-middle	End-to-end signed mandates

5.3 Verification Flow

```
def verify_mandate(mandate_vc: dict) -> bool:
    # 1. Extract issuer DID
```

```
    issuer_did = mandate_vc["issuer"]

    # 2. Resolve DID document
    did_document = resolve_did(issuer_did)

    # 3. Get public key from verification method
    public_key = did_document["verificationMethod"][0]["publicKeyHex"]

    # 4. Verify signature
    proof = mandate_vc["proof"]
    signature = decode_jws(proof["jws"])

    return verify_signature(
        message=mandate_vc["credentialSubject"],
        signature=signature,
        public_key=public_key
    )
```

6. Implementation Considerations

6.1 Minimum Viable Components

To implement Agentic Commerce, you need:

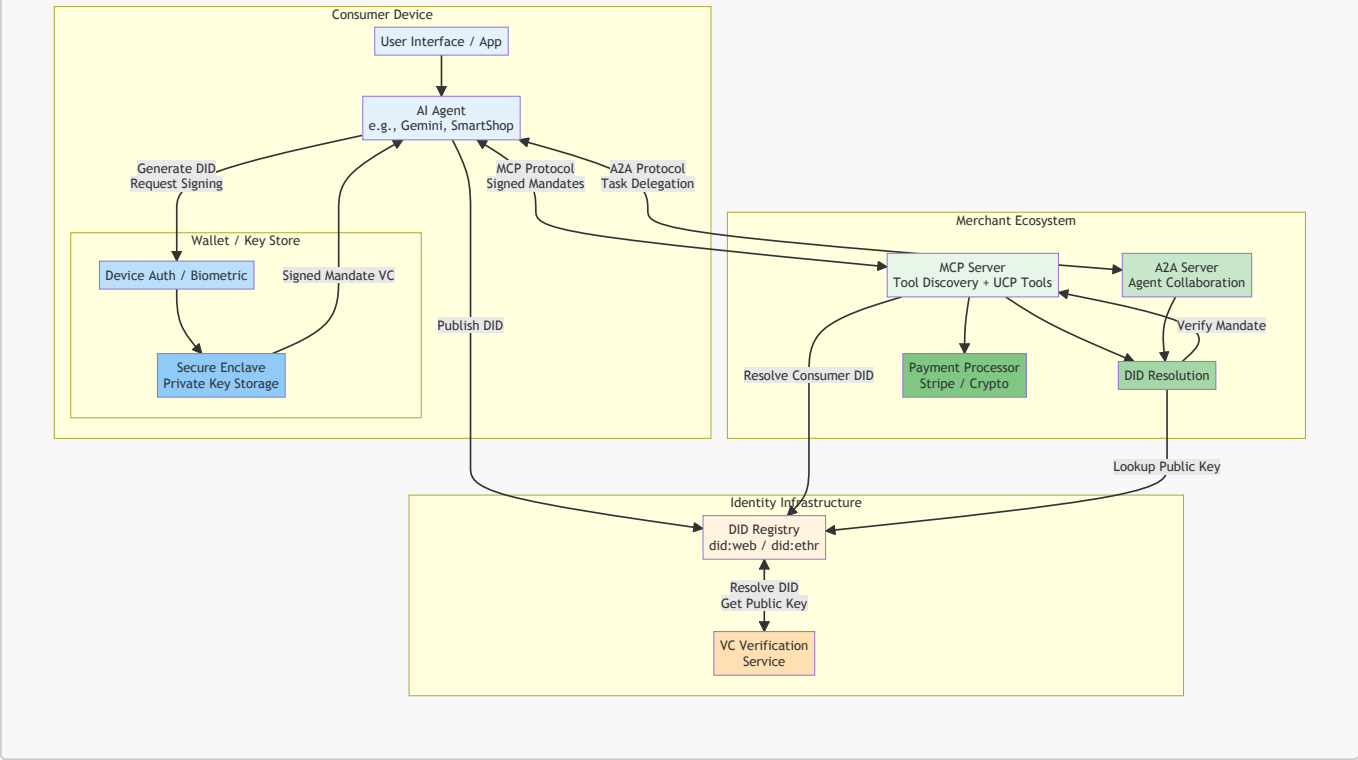
Component	Responsibility
Identity Service	DID creation, key management, VC signing/verification
MCP Server	Tool exposure, UCP tool handlers
Consumer Agent	Natural language processing, mandate orchestration
Storage	Mandate persistence, session state

6.2 Technology Choices

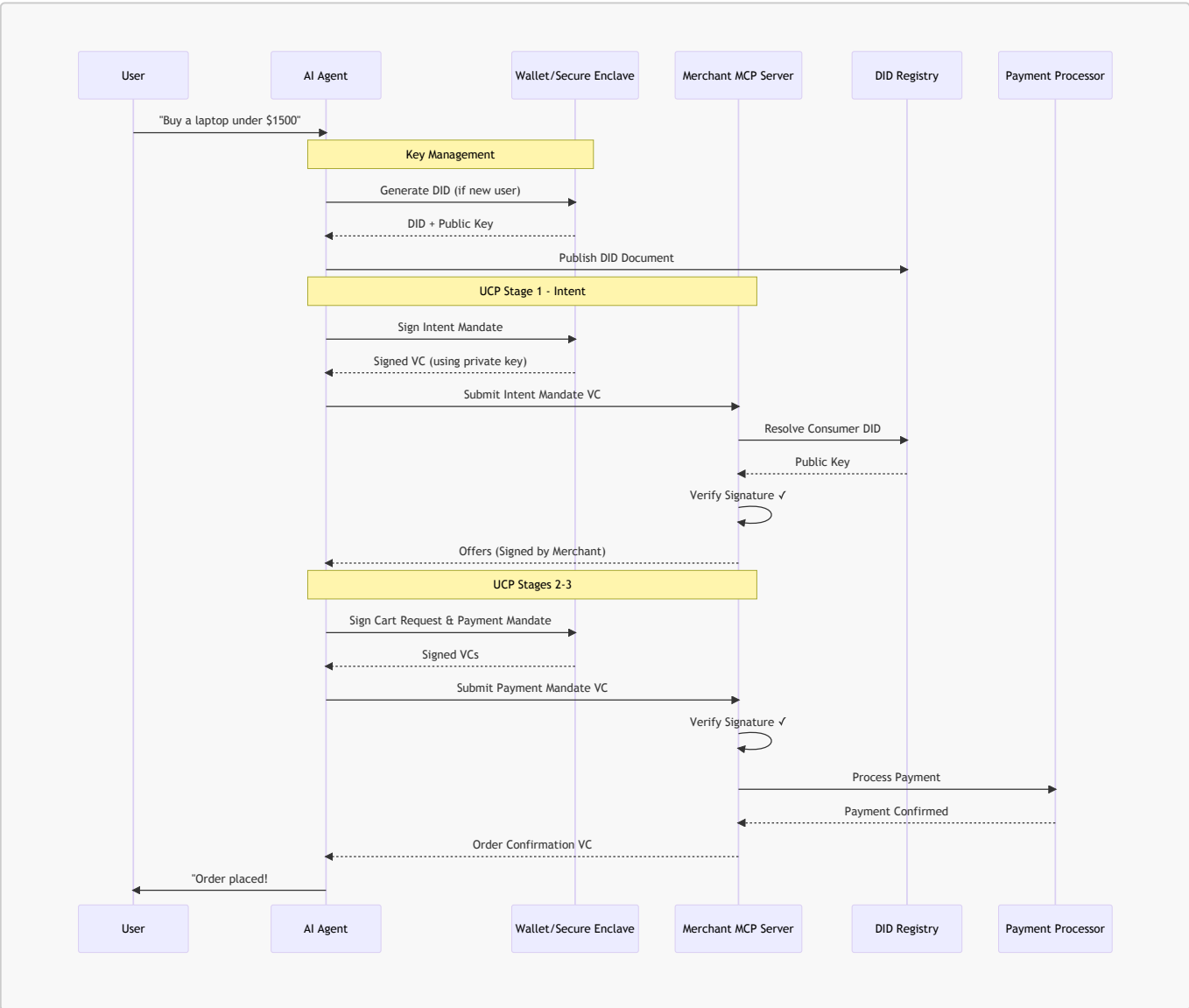
Layer	Options
DID Method	did:web (simple), did:key (no infra), did:ethr (decentralized)
Signatures	EcdsaSecp256k1, Ed25519
Transport	HTTP/JSON-RPC (MCP), WebSockets (real-time)
Agent Framework	LangChain, LangGraph, custom

6.3 Deployment Architecture

The architecture spans three ecosystems: Consumer (local device), Identity Infrastructure (third-party or self-hosted), and Merchant Services.



6.4 Data Flow: Signing and Verification



6.5 Component Responsibilities

Component	Location	Responsibilities
AI Agent	Consumer Device	Natural language understanding, UCP orchestration, mandate creation
Wallet / Key Store	Consumer Device	DID generation, private key storage, mandate signing
Secure Enclave	Consumer Device	Hardware-protected key storage, biometric authentication
DID Registry	Third-Party / Self-hosted	DID document storage, public key resolution
MCP Server	Merchant Infrastructure	Tool discovery, UCP handler, signature verification
A2A Server	Merchant Infrastructure	Agent collaboration, task delegation
Payment Processor	Merchant Infrastructure	Payment rail integration (Stripe, crypto, etc.)

6.6 Key Design Principles

- Keys Never Leave the Device:** Private keys are stored in the secure enclave; only signatures are transmitted
- Wallet-Based Signing:** All mandates are signed locally before transmission
- DID Resolution for Trust:** Merchants resolve consumer DIDs to verify signatures; consumers resolve merchant DIDs to verify offers
- Protocol Separation:** MCP for tool discovery and invocation; A2A for complex multi-agent workflows; UCP for commerce-specific handshake

7. The Path Forward

7.1 Open Challenges

Payment Rails Integration: Bridging cryptographic mandates to traditional payment networks (card, ACH, wire) and cryptocurrency networks requires careful design.

Regulatory Compliance: Agent-conducted commerce must navigate consumer protection, KYC/AML, and jurisdiction-specific regulations. The non-repudiable nature of signed mandates provides clear audit trails.

Discovery & Registry: How do agents find merchants? Options include decentralized registries, curated directories, or web-based discovery.

User Experience: Agents need intuitive interfaces for preference setting, approval workflows for high-value purchases, and transparency into agent actions.

7.2 Adoption Path

1. **Developer Tools:** SDKs and libraries for easy integration
 2. **Reference Implementations:** Open-source examples demonstrating the full stack
 3. **Merchant Onboarding:** Simple paths for businesses to expose MCP/UCP endpoints
 4. **Agent Ecosystem:** Growing network of compatible consumer and service agents
-

Conclusion

This technical architecture provides the blueprint for implementing Agentic Commerce:

- **DIDs and VCs** establish decentralized, verifiable identity
- **MCP** enables universal tool discovery and invocation
- **UCP** defines the three-mandate commerce workflow with bidirectional trust
- **A2A** enables multi-agent collaboration for complex transactions

Together, these protocols create an open, secure foundation for AI agent commerce—without centralized intermediaries.

Part 3 of this whitepaper series will provide a complete implementation walkthrough with working code examples.

Acknowledgments

This work builds upon open protocols and standards developed by leading organizations:

- **UCP** (Unified Commerce Protocol) — Google
- **MCP** (Model Context Protocol) — Anthropic
- **A2A** (Agent-to-Agent Protocol) — Google
- **DIDs & VCs** — W3C

Feedback Welcome: Community feedback and contributions are encouraged.